Ship a Next.js App with Claude Code

A hands-on workshop — build a full-stack app with an agentic coding tool



Maurice de Beijer

The Problem Solver

Freelance developer / instructor

Twitter/X: @mauricedb
Web: theproblemsolver.dev
Email: maurice.de.beijer@gmail.com



What We'll Build Today

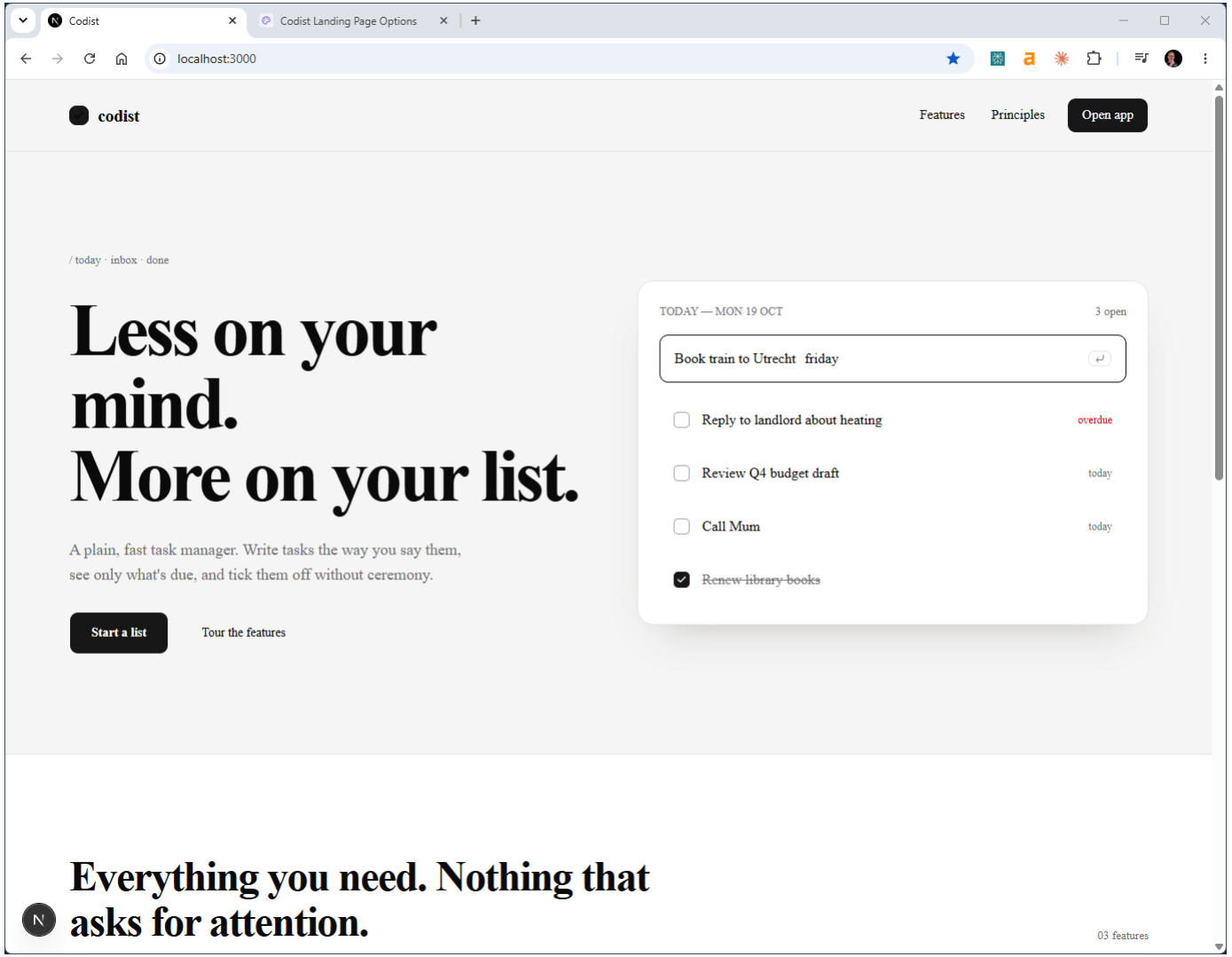
What we're building — Codist

- A full-stack task management app
- Landing page, Inbox view, Today view
- Real database with Prisma + SQLite
- Polished UI with shadcn/ui components

What you'll master

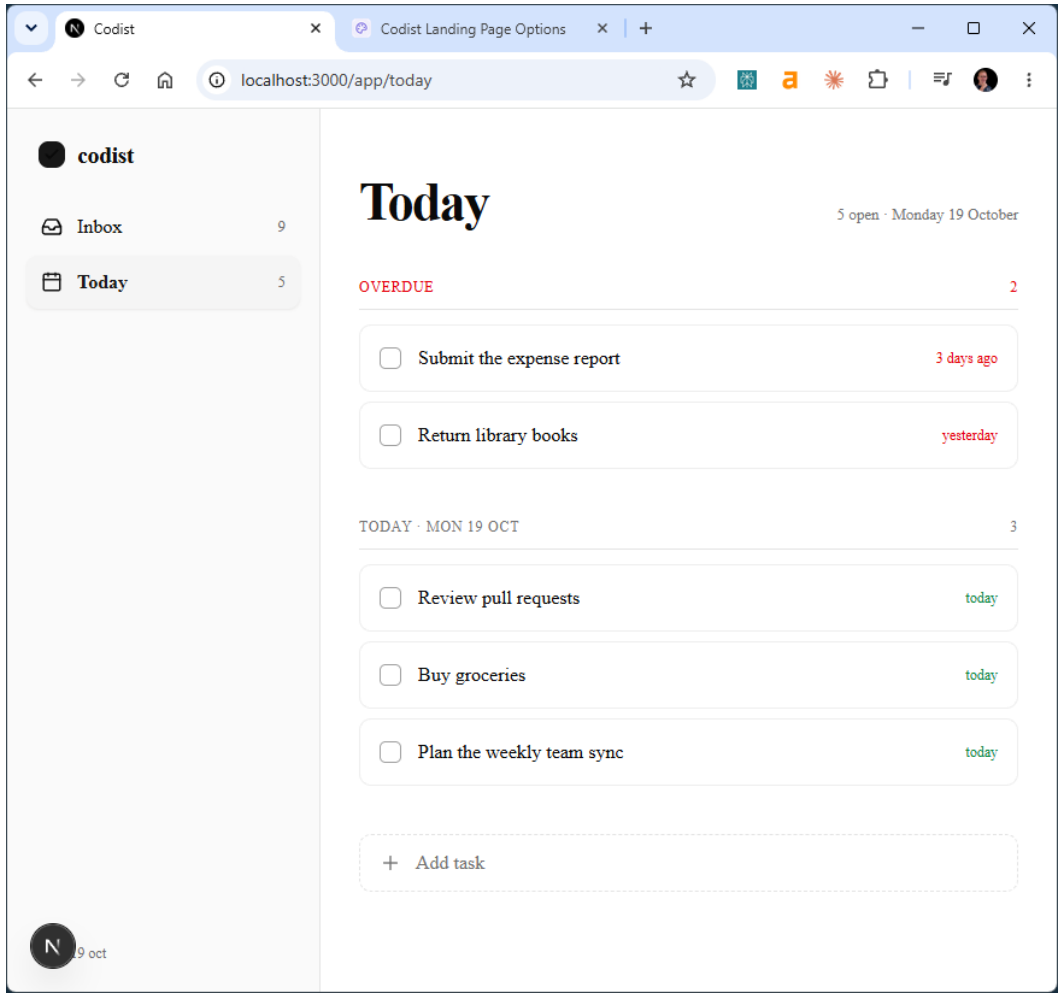
- **Agentic execution** — multi-file changes from one prompt
- **CLAUDE.md** — persistent project memory & conventions
- **Full-stack scaffolding** — schemas & server actions in minutes
- **UI iteration** — from prompt to polished interface

The Landing Page We Will Build



Preview — the finished Codist landing page

The Task Pages We Will Build



Preview — Today and Inbox views with the nav sidebar

Prerequisites

- [ ] Install Node.js & npm
- [ ] Clone the GitHub repositories
- [ ] Install Claude Code

Following Along

Repo with start code:

github.com/mauricedb/codist

Workshop resources:

github.com/mauricedb/codist-workshop

Install Node.js & npm & Git

YOUR TURN · 1 MIN

```
node --version
git --version
npm --version
```

Grab Node.js from nodejs.org and Git from git-scm.com if either command comes back empty.

Install Claude Code

YOUR TURN · 5 MIN

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Windows CMD
curl -fsSL https://claude.ai/install.cmd -o install.cmd &&
install.cmd && del install.cmd

# npm alternative (Node.js 18+)
npm install -g @anthropic-ai/claude-code

claude --version
claude doctor
```

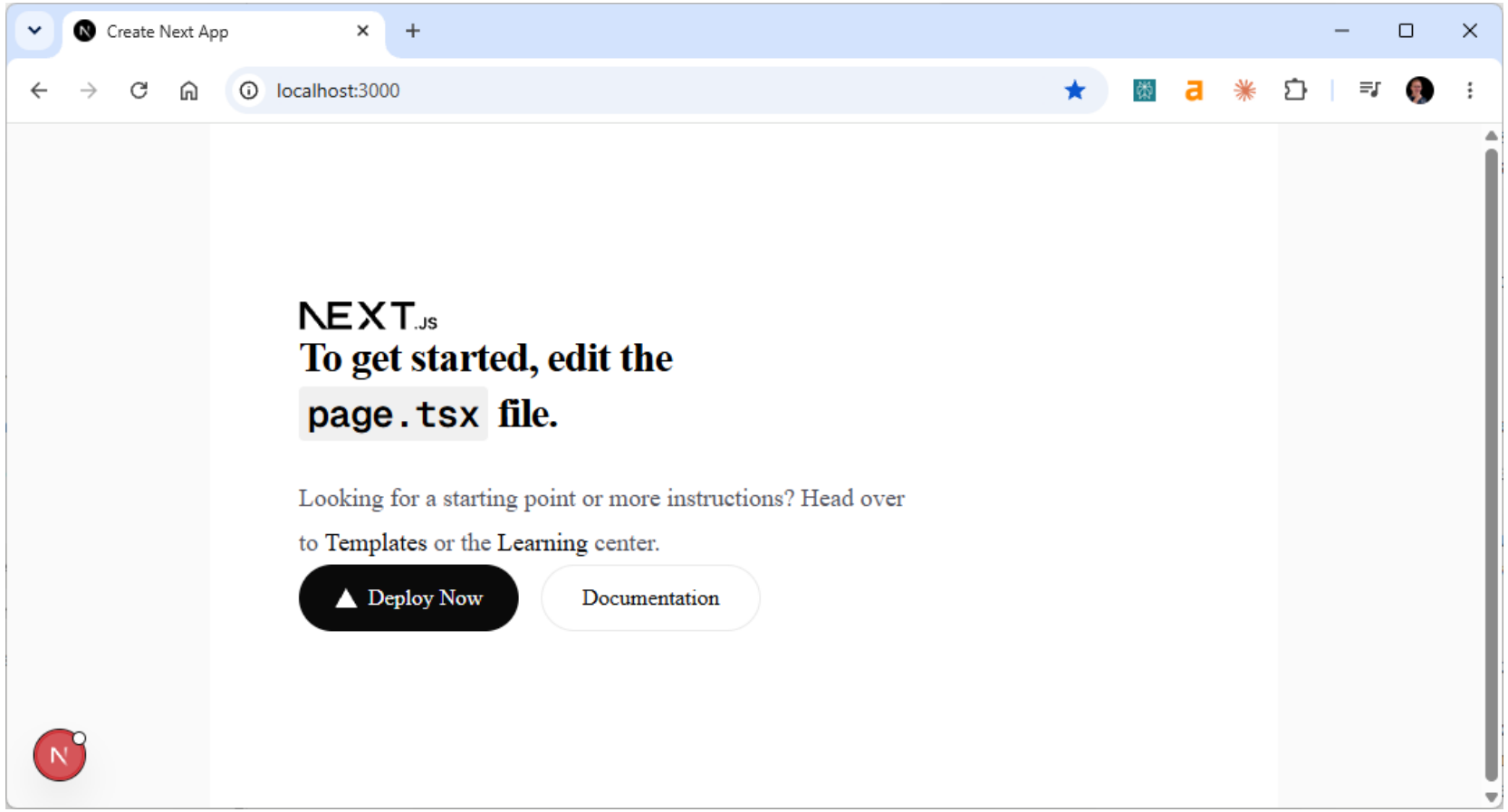
First run opens a browser to log in — a Claude.ai Pro/Max/Team/Enterprise or Anthropic Console account is required (not included on the free tier).

Install the Start Repository

```
git clone https://github.com/mauricedb/codist.git
cd codist
npm install
npm run dev
```

The Application

YOUR TURN · 4 MIN



Confirm the default Next.js starter page loads at localhost:3000



What the Starter Repo Includes

- **A new Next.js app with default settings**
  - npx create-next-app@latest codist --yes
- **Prisma with SQLite**
  - npx prisma init --datasource-provider sqlite --output ../src/generated/prisma
- **shadcn/ui, pre-loaded with what Codist needs**
  - npx shadcn@latest init
  - npx shadcn@latest add badge button calendar card checkbox input label popover separator

Intro to Claude Code

SECTION 2

What is Claude Code?

- **An agentic coding tool that runs in your terminal** — also a VS Code/JetBrains extension, desktop app, and web
- **Builds features from plain-English descriptions** — plans, writes, and checks the code runs
- **Debugs** — investigates the codebase to find and fix a bug you describe
- **Navigates any codebase** — full project awareness, plus the web and MCP-connected tools
- **Takes direct action** — edits files, runs commands, creates commits



The Shift in How We Code

Before — manual, file-by-file

- Write every line, search docs, hold context in your head

With Claude Code

- Describe the outcome, let it read the codebase and execute across files
- **Hand off tedious work** — lint fixes, merge conflicts, release notes
- **Scriptable, Unix-style** — pipe logs straight into a prompt



Core Features

- **CLAUDE.md** — project memory, loaded every session
- **Plan Mode** — research & propose a plan before touching files
- **Slash commands** — reusable custom commands
- **Subagents** — isolated context for parallel work
- **Hooks** — deterministic automation on events
- **MCP servers** — connect external tools & data



Talking to Claude Code

"@app/(tasks)/today/page.tsx —

why does the sort break

on an empty task list?"

Reference a file inline with @ — opens a fuzzy file finder.



Agentic Execution

- **Scaffold a full Next.js route** — page, layout, loading, error — from one description
- Generate typed API routes and server actions end-to-end
- Apply the same change across many files at once
- **You stay in control** — review the diff, or start in Plan Mode for anything higher-stakes

CLAUDE.md — Project Memory

```
# Codist
Next.js 15 + TypeScript + Prisma
+ SQLite + Tailwind + shadcn/ui

## Conventions
- Strict TypeScript, no any
- Validate input with Zod

## Commands
- npm run dev
- npm run lint
```

A markdown file Claude loads automatically before your first prompt — same role as Cursor Rules. Conventions apply to every future session.



Claude Code + Next.js

- **Understands App Router conventions**
  - Scaffold a route, get page/layout/loading/error together
- **Great for server actions & typed API routes**
  - Describe the endpoint, get validated code
- **UI iteration is conversational**
  - Describe a component, get Tailwind-styled, accessible JSX



The Agentic Dev Loop

1. Define

– Write CLAUDE.md, tell Claude your stack

2. Scaffold

– Describe a feature, generate routes & components

3. Iterate

– Refine logic and UI through follow-ups

4. Harden

– Ask for tests, edge cases, a cleanup pass

5. Ship

– Review the diff, commit, repeat

Quick Setup Check

YOUR TURN · 15 MIN

- [ ] Claude Code installed — claude --version
- [ ] Logged in — claude doctor shows no issues
- [ ] Workshop repo cloned
- [ ] claude starts a session cleanly in the repo root

Raise your hand if anything's missing — we build in 15 minutes!

Break time



15 minutes — grab a coffee, stretch, ask a question

Building the Home Page

CLAUDE.md doesn't exist yet — that's the point

SECTION 3

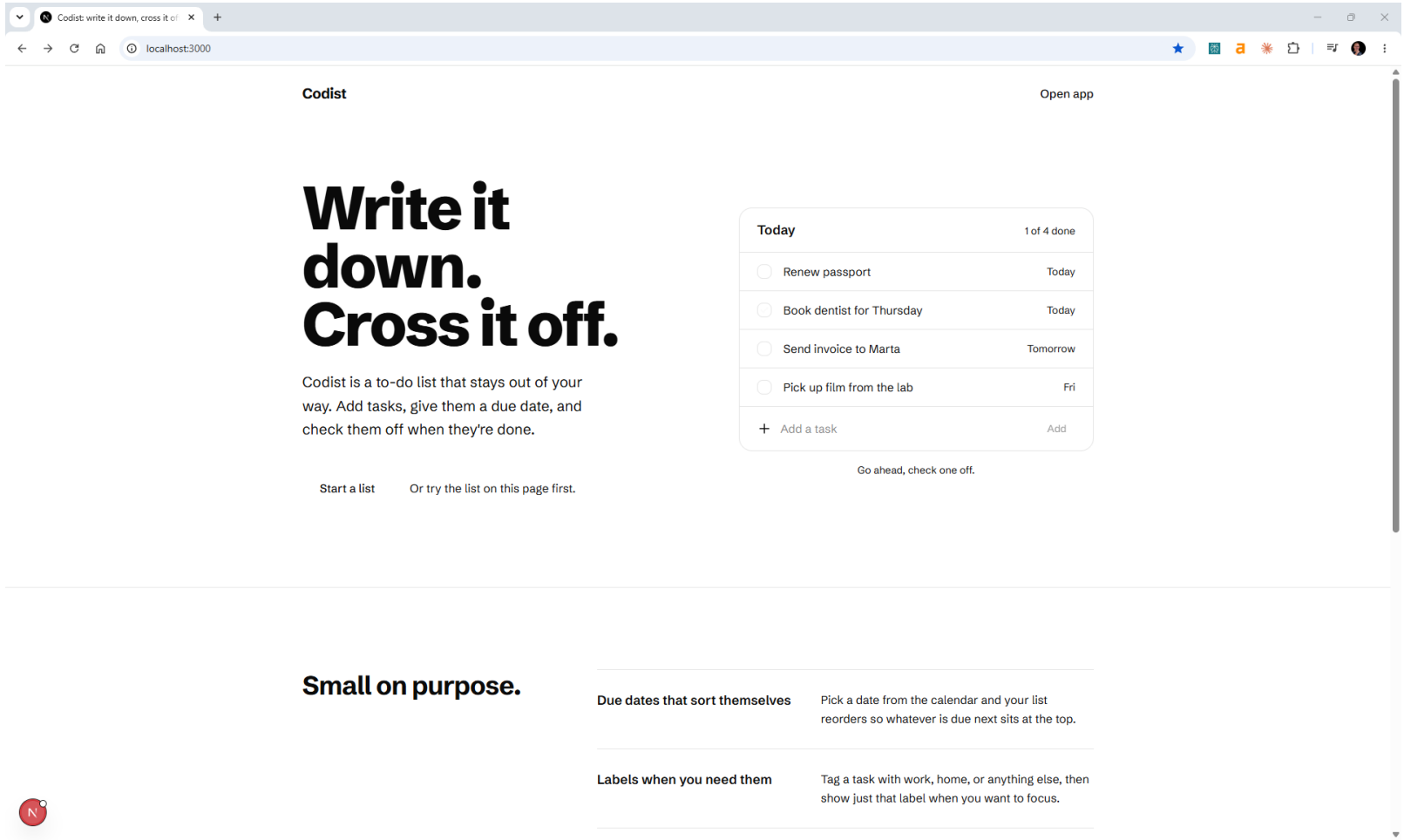


A Very Naïve Prompt

"Build me a new landing page for a to-do application named Codist."

The Result

YOUR TURN · 4 MIN



Functional but generic — plain HTML, no shadcn/ui, no real brand tone

A Better Prompt

Replace the default Next.js landing page (app/page.tsx) with a Codist landing page, a to-do app, that matches the attached reference image @docs/images/1-todoist-landing-page.jpg.

First read: the image, the current page, globals.css / tailwind config (design tokens), and components/ui. List the sections you'll build before editing.

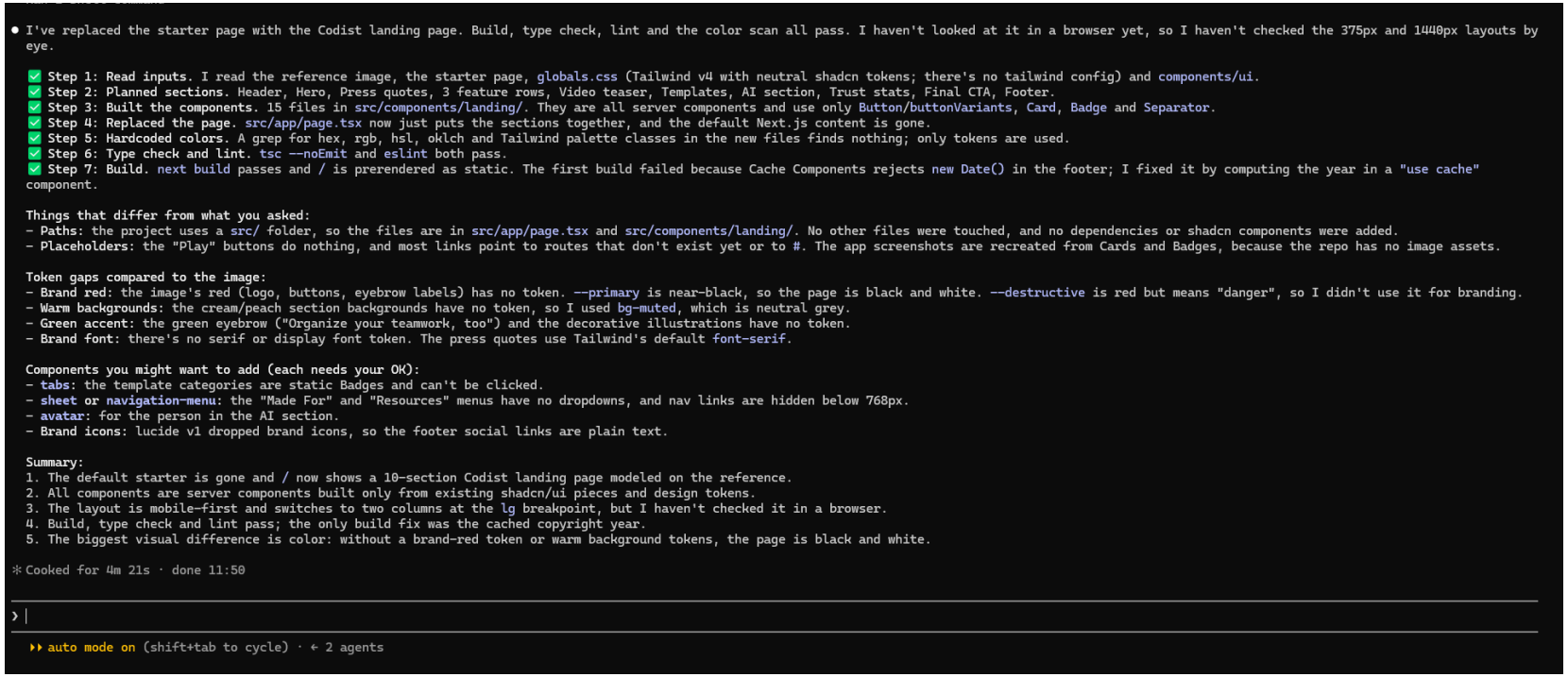
Rules:
- Use only existing shadcn/ui components.
- Use only our design tokens (bg-background, text-primary, ...).
  NEVER hardcode colors. Report any token gaps vs. the image.
- Responsive (375px and 1440px), server components by default.
- Edit only app/page.tsx and components/landing/. Ask before adding
  dependencies, adding shadcn components, or touching other files.

Done when:
- / renders a landing page matching the image's sections and layout.
- Default Next.js starter content is gone.
- Build, lint and type check pass.
- No hardcoded color values in the new files.

Report each step with ✅, then a 5-line summary.

Tip: add "Start the dev server in the background." *Claude runs it non-blocking, so it can preview its own changes while you keep chatting.*

The Response



● I've replaced the starter page with the Codist landing page. Build, type check, lint and the color scan all pass. I haven't looked at it in a browser yet, so I haven't checked the 375px and 1440px layouts by eye.

✅ **Step 1: Read inputs.** I read the reference image, the starter page, `globals.css` (Tailwind v4 with neutral shadcn tokens; there's no tailwind config) and `components/ui`.
✅ **Step 2: Planned sections.** Header, Hero, Press quotes, 3 feature rows, Video teaser, Templates, AI section, Trust stats, Final CTA, Footer.
✅ **Step 3: Built the components.** 15 files in `src/components/landing/`. They are all server components and use only `Button/buttonVariants`, `Card`, `Badge` and `Separator`.
✅ **Step 4: Replaced the page.** `src/app/page.tsx` now just puts the sections together, and the default Next.js content is gone.
✅ **Step 5: Hardcoded colors.** A grep for hex, rgb, hsl, oklch and Tailwind palette classes in the new files finds nothing; only tokens are used.
✅ **Step 6: Type check and lint.** `tsc --noEmit` and `eslint` both pass.
✅ **Step 7: Build.** `next build` passes and `/` is prerendered as static. The first build failed because Cache Components rejects `new Date()` in the footer; I fixed it by computing the year in a `"use cache"` component.

Things that differ from what you asked:
- **Paths:** the project uses a `src/` folder, so the files are in `src/app/page.tsx` and `src/components/landing/`. No other files were touched, and no dependencies or shadcn components were added.
- **Placeholders:** the "Play" buttons do nothing, and most links point to routes that don't exist yet or to `#`. The app screenshots are recreated from Cards and Badges, because the repo has no image assets.

Token gaps compared to the image:
- **Brand red:** the image's red (logo, buttons, eyebrow labels) has no token. `--primary` is near-black, so the page is black and white. `--destructive` is red but means "danger", so I didn't use it for branding.
- **Warm backgrounds:** the cream/peach section backgrounds have no token, so I used `bg-muted`, which is neutral grey.
- **Green accent:** the green eyebrow ("Organize your teamwork, too") and the decorative illustrations have no token.
- **Brand font:** there's no serif or display font token. The press quotes use Tailwind's default `font-serif`.

Components you might want to add (each needs your OK):
- **`tabs`:** the template categories are static Badges and can't be clicked.
- **`sheet` or `navigation-menu`:** the "Made For" and "Resources" menus have no dropdowns, and nav links are hidden below 768px.
- **`avatar`:** for the person in the AI section.
- **Brand icons:** lucide v1 dropped brand icons, so the footer social links are plain text.

Summary:
1. The default starter is gone and `/` now shows a 10-section Codist landing page modeled on the reference.
2. All components are server components built only from existing shadcn/ui pieces and design tokens.
3. The layout is mobile-first and switches to two columns at the `lg` breakpoint, but I haven't checked it in a browser.
4. Build, type check and lint pass; the only build fix was the cached copyright year.
5. The biggest visual difference is color: without a brand-red token or warm background tokens, the page is black and white.

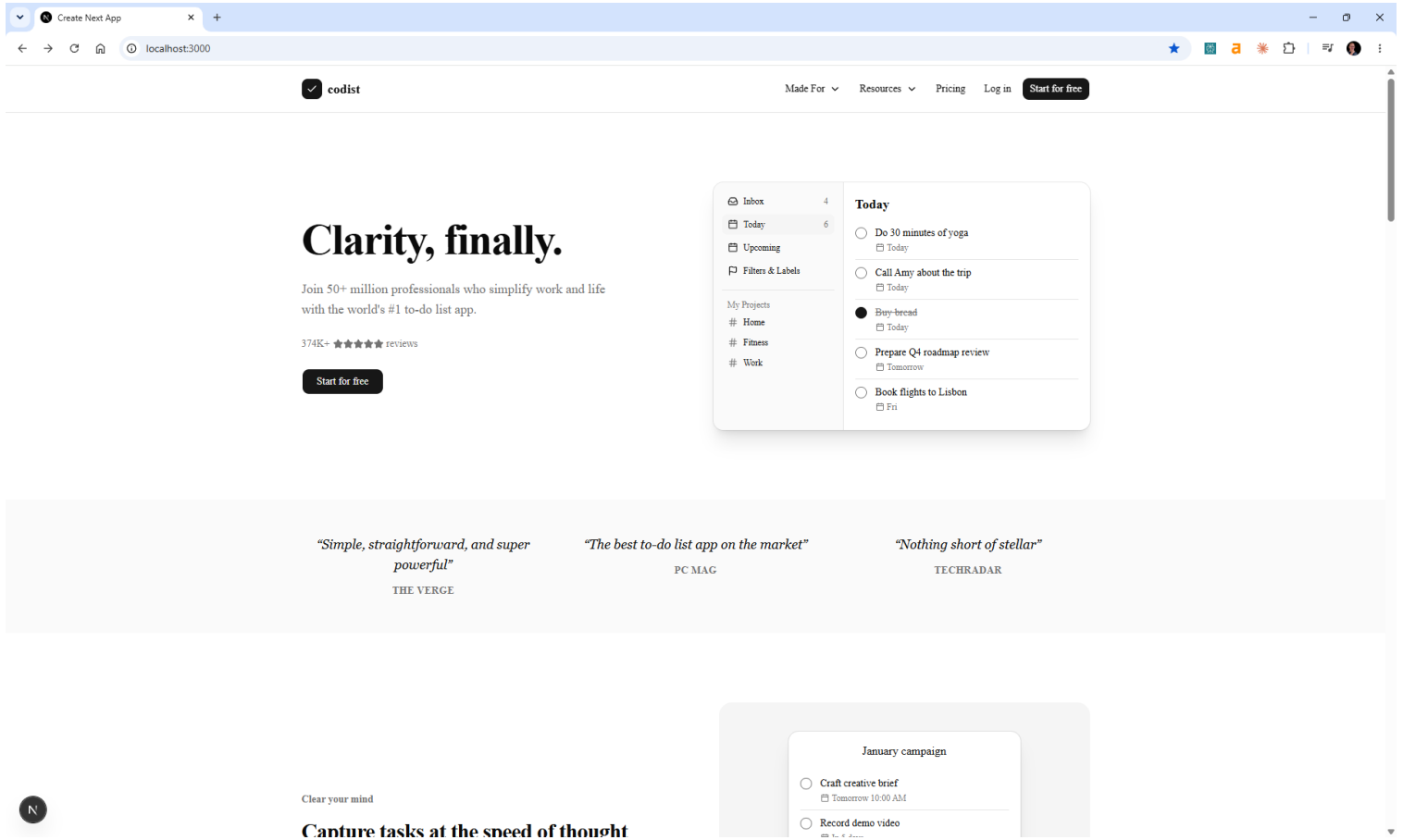
✻ Cooked for 4m 21s · done 11:50

›

⏵⏵ auto mode on (shift+tab to cycle) · ← 2 agents

The Result

YOUR TURN · 5 MIN



Much closer — shadcn components in use, layout matches the reference

CLAUDE.md Files

SECTION 4

Project-Specific CLAUDE.md

Root CLAUDE.md

- High-level stack, conventions, and commands
- */init*
  Initialize a new CLAUDE.md file with codebase documentation

.claude/rules/ — path-specific

- **prisma.md, shadcn.md, tailwind.md** — only load when Claude touches a matching file

Where to find ready-made ones

- https://www.skills.sh
- https://github.com/hesreallyhim/awesome-claude-code
- https://github.com/abhishekray07/claude-md-templates

Generating CLAUDE.md with /init

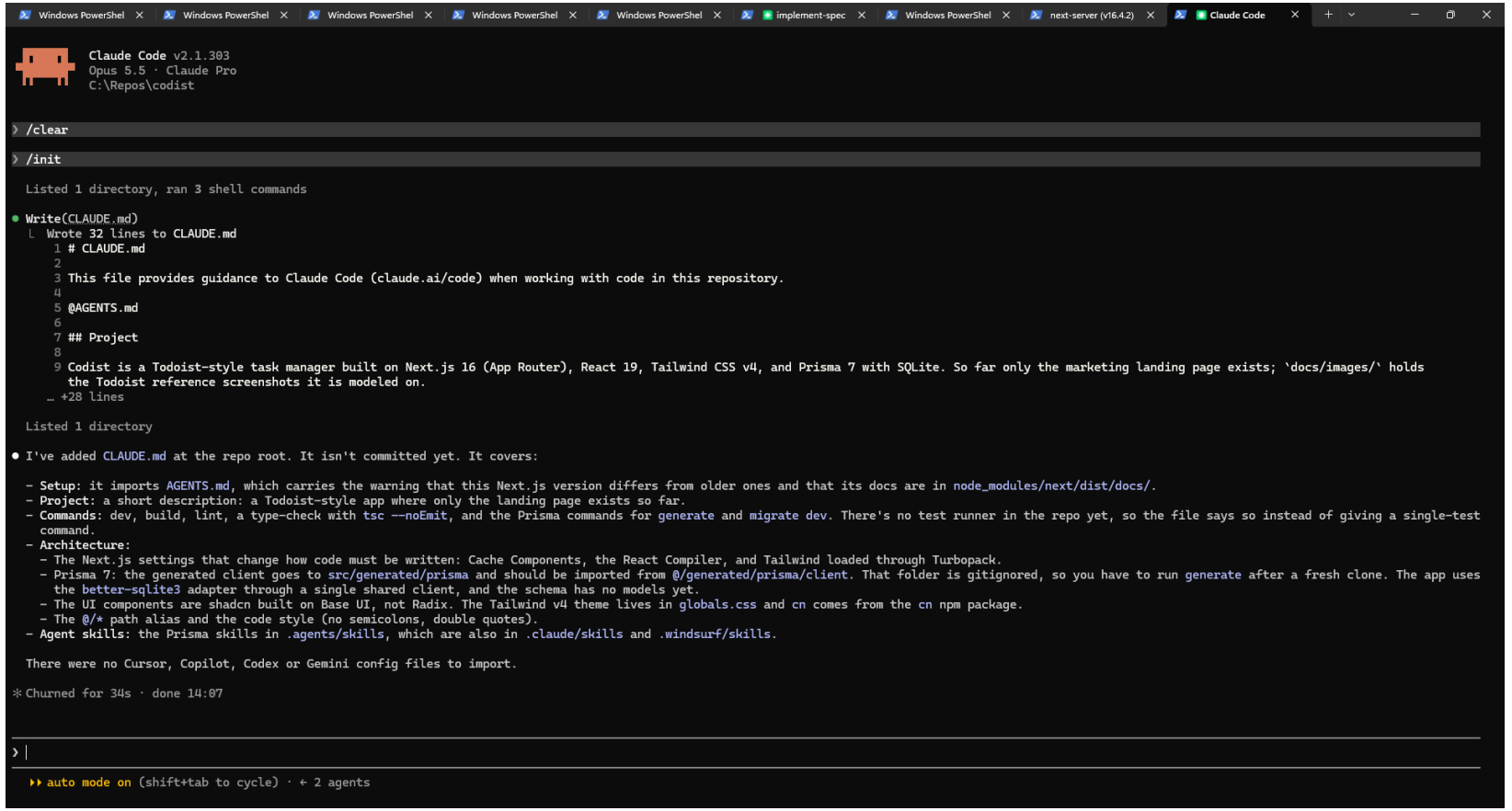
```
/init
```

Claude scans the file tree, package.json, and existing code to draft a baseline CLAUDE.md — and typically asks a few clarifying questions first. Always review before committing.

/init's exact behavior has changed across releases

The Response

YOUR TURN · 4 MIN





Application Architecture Guidance

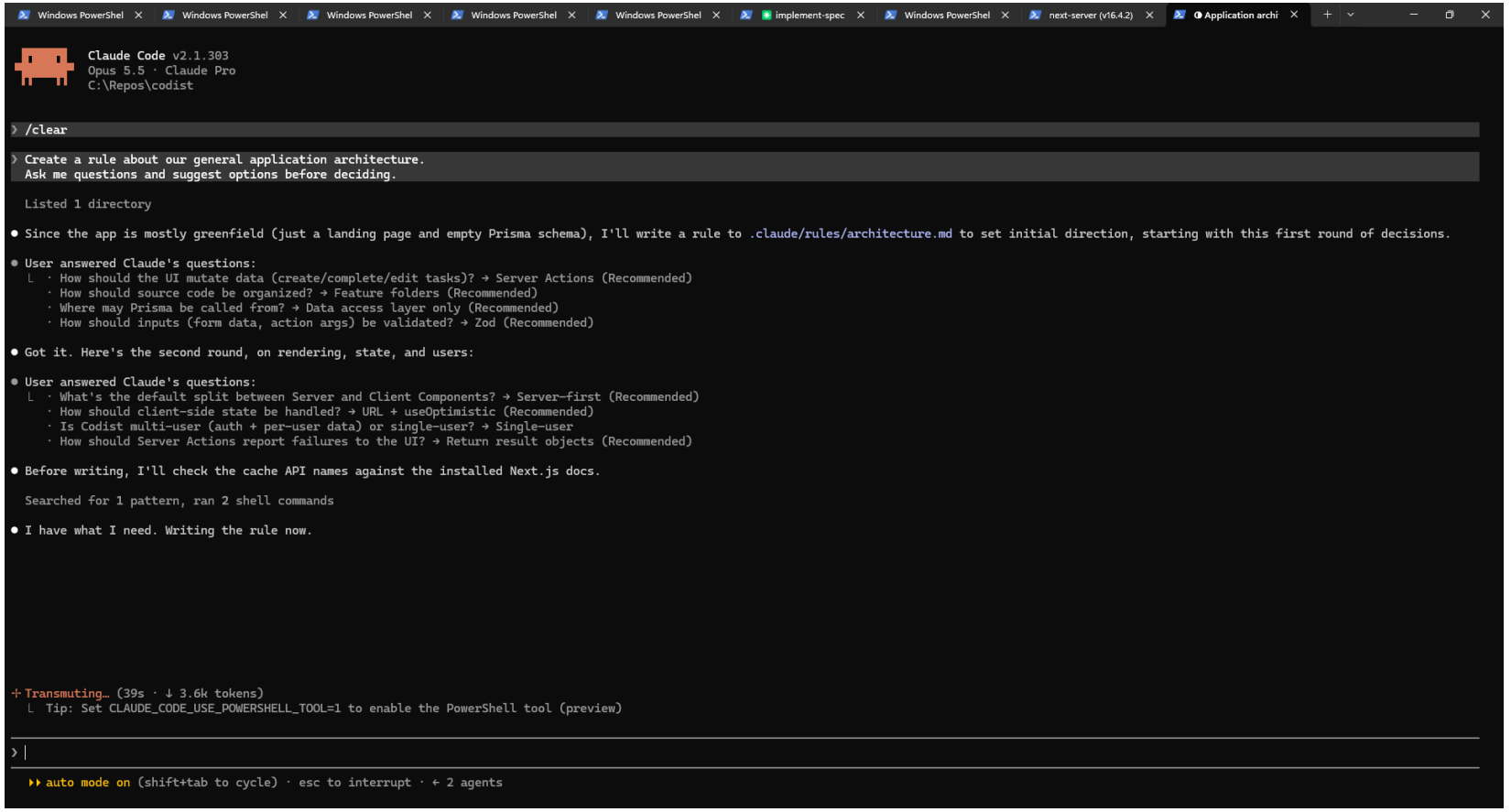
"Create a rule about our general application architecture.

Ask me questions and suggest options before deciding."

Claude interviews you, then writes the result to .claude/rules/architecture.md — applied automatically from then on.

The Response

YOUR TURN · 6 MIN



My Claude Code Workflow

CLAUDE.md is in place — nothing below restates conventions

SECTION 5

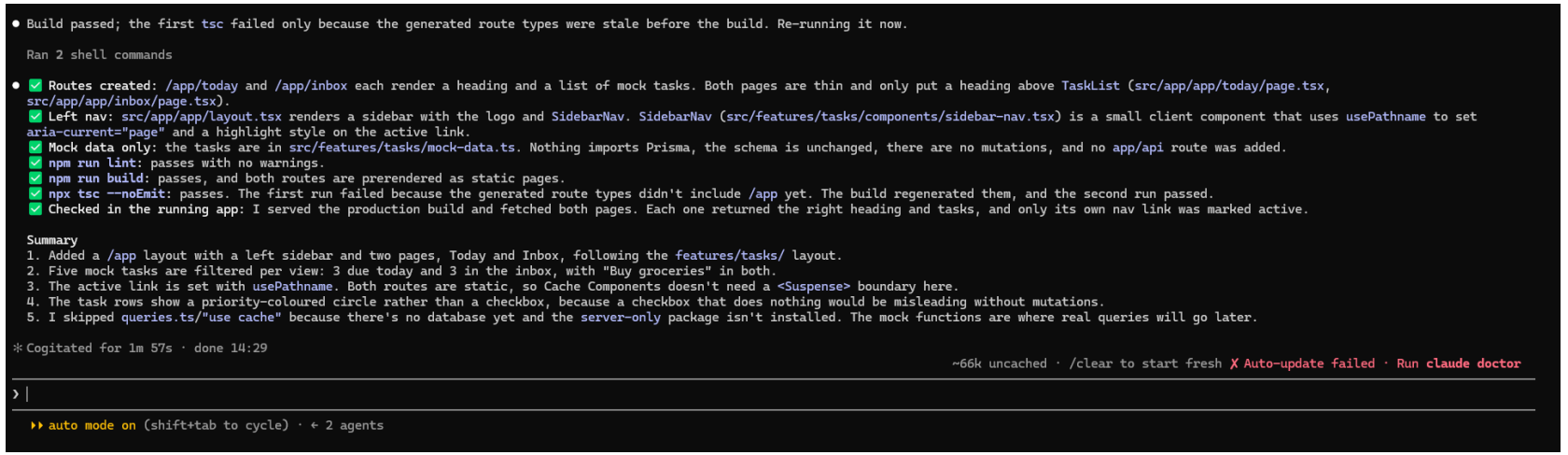
The First Prompt

"Create the routes /app/today and /app/inbox, with a left nav that switches between them. Use mock data only: no Prisma, no schema changes, no mutations.

Done when:
- /app/today and /app/inbox each render a heading and their mock tasks.
- The left nav links both routes and highlights the active one.
- No Prisma import or API route was added.
- `npm run build`, `npm run lint` and `npx tsc --noEmit` pass.

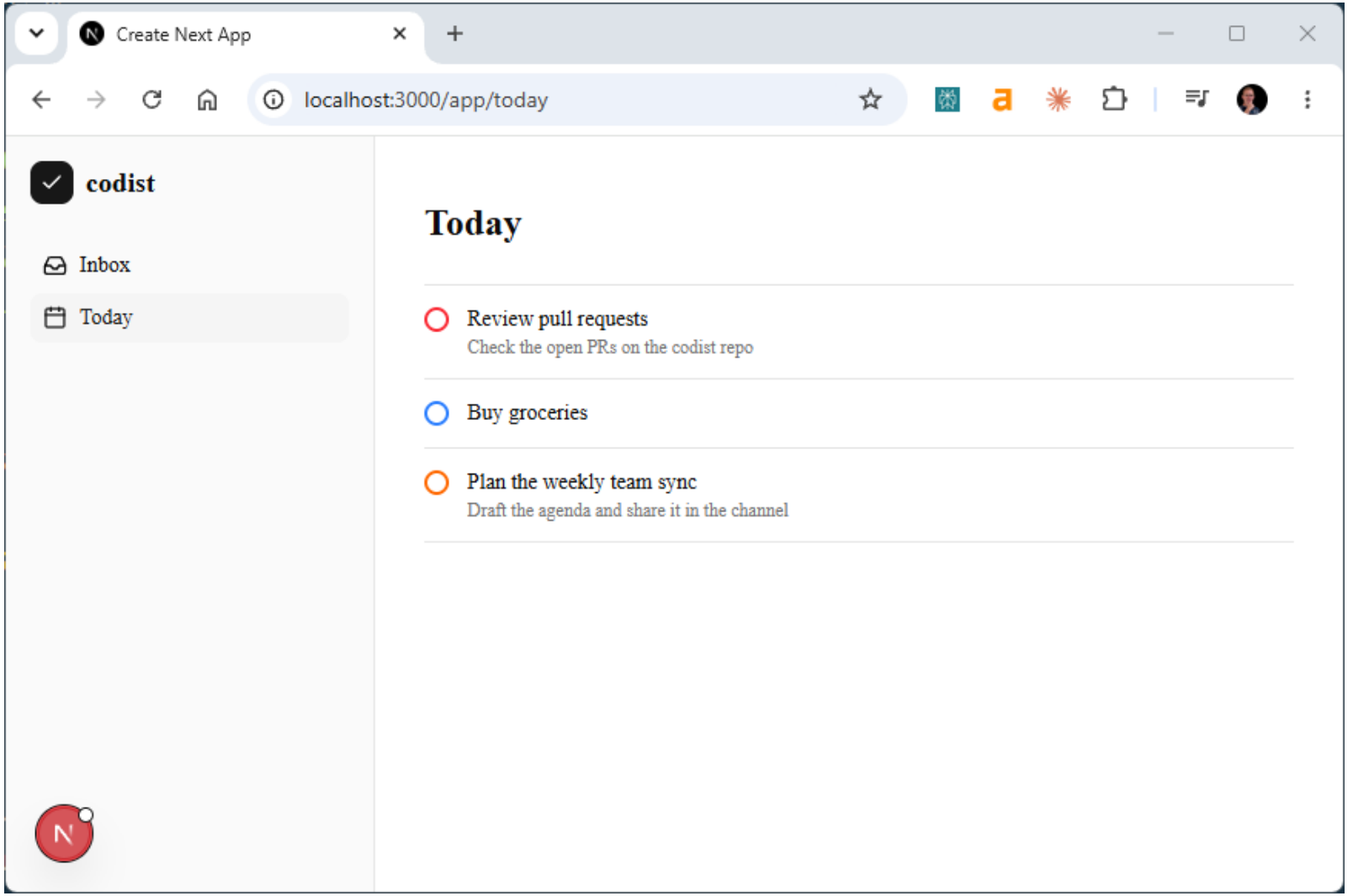
Report each step with ✅, then a 5-line summary."

The Response



The Result

YOUR TURN · 6 MIN



Nav on the left, Today view showing the mock tasks

Prompt to Add Details

"Add due-date status to the task rows on /app/today and /app/inbox, and a checkbox to every row to mark items as done.

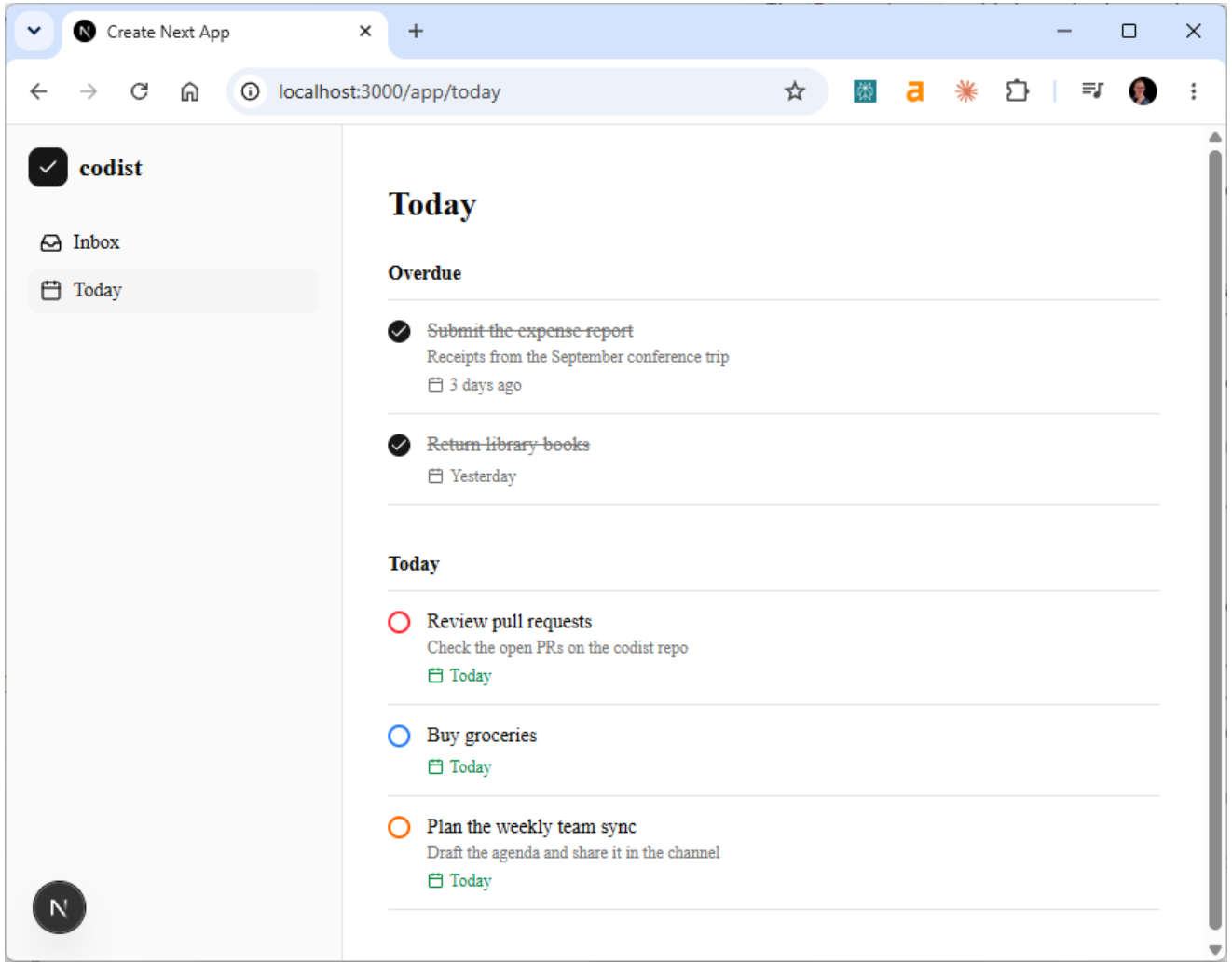
- Overdue tasks are shown in red, due-today tasks in green.
- Add overdue mock tasks too (Today lists them above today's tasks, as in the Todoist screenshots).
- The checkbox toggles with local UI state only: no server action and no persistence yet.
- Red uses the destructive token. If there is no token for green, stop and ask before adding one to globals.css.
- Status must not rely on color alone: include the date label as text.

Done when:
- Overdue rows render in red and due-today rows in green, in both light and dark mode.
- Every row has an accessible checkbox that toggles its checked state and shows a completed style (e.g. strikethrough, muted).

Report each step with ✅, then a 5-line summary."

The Result

YOUR TURN · 6 MIN



Overdue in red, due-today in green, checkboxes on every row



Adding a Database with Prisma

"Replace the mock tasks with a real Task table in SQLite, and make /app/today and /app/inbox read from it.

- Add a Task model only: no Project model, no userId. Fields: title, due date (optional, a calendar date, not a timestamp), completedAt (optional), createdAt. Create it with a named migration.
- Seed it from the current mock data via `npx prisma db seed`. Seed due dates relative to the day you run it (e.g. today minus 2 days) so the overdue / due-today demo never goes stale. The seed must be re-runnable without duplicating rows.
- Replace the mock data in queries.ts with Prisma reads that return the same shape the pages already use, so the components don't change. Today = incomplete tasks due today or earlier, Inbox = all incomplete tasks.
- "Today" is the user's local calendar day (Europe/Amsterdam), not UTC. Compute it in a way that works with Cache Components.

Done when:
- The Task migration exists and applies cleanly on a fresh database.
- `npx prisma db seed` fills the table and can be run twice without duplicates.
- /app/today and /app/inbox render the seeded rows, with overdue rows red and due-today rows green exactly as before.
- Changing a task's due date directly in the database changes which page and color it shows, after a refresh.
- No mock data is left in queries.ts, and no page, component, or action imports Prisma."



Seed and Inspect the Database

```
npx prisma db seed
npx prisma studio
```

The Response

YOUR TURN · 3 MIN

● Both pages now read from a real SQLite Task table, and every "done when" item checks out against a production build.

✅ **Schema and migration**: added a Task model with id, title, dueDate, completedAt and createdAt. dueDate is stored as text in "YYYY-MM-DD" form, so it's a plain calendar date and easy to edit by hand. The migration is prisma/migrations/20261019140539_add_task, and it applied cleanly to a brand-new database with migrate deploy.

✅ **Seed**: npx prisma db seed runs prisma/seed.ts through tsx, which I added as a dev dependency. Due dates are set relative to today in Amsterdam (−3, −1, 0, 0, 0, none, none). Each row has a fixed id and is upserted, so after two runs the table still holds 7 rows; a re-run also moves the dates forward to the current day.

✅ **Queries**: the new src/features/tasks/queries.ts returns the same task shape as before. Today shows open tasks due before or on today, split into Overdue and Today sections; Inbox shows all open tasks. mock-data.ts is deleted. Its date helpers moved to due-date.ts, and the only change in the two components is where they import from. Only queries.ts and src/lib/prisma.ts import Prisma.

✅ **"Today" with Cache Components**: getToday() in today.ts waits on io(), reads the clock and returns the Amsterdam date. That date is passed into the cached queries as an argument, so the cache rolls over at local midnight. Both pages wrap the data in <Suspense> with a new TaskListSkeleton.

✅ **Checks**: tsc, lint and build all pass. On the production server, overdue rows rendered red and due-today rows green. I then changed rows directly in SQLite: I set one task's due date to today and another's to 5 days ago, and marked a third done. Each moved to the right page with the right color after a refresh, and I reseeded the data afterwards.

Things to know:
- **Priority and description**: the model you specified has no priority or description fields, so every task shows the "no priority" grey ring and no description. Before, some tasks had red, orange or blue rings and a description line.
- **Delay after a database edit**: I set cacheLife("seconds") because nothing invalidates the cache when the database is edited outside the app. The first refresh within about a second of an edit can still show the old data; the next refresh shows the new data. Changes made later through Server Actions will use updateTag("tasks") and show up immediately.
- **dev.db is tracked in git**: seeding modified it. You may want to add it to .gitignore and stop tracking it (git rm --cached dev.db). I haven't done that, and I haven't committed anything.

Task table + named migration replace mock data; applies cleanly on a fresh DB.
Idempotent seed (upsert by fixed id) with due dates relative to today in Amsterdam.
queries.ts returns the old shape, so components only changed an import path.
Today = io() + Amsterdam calendar date, passed as an argument (part of the cache key).
Verified: tsc/lint/build pass; colors are correct; direct DB edits show after a refresh.

✻ Churned for 3m 57s · done 16:08



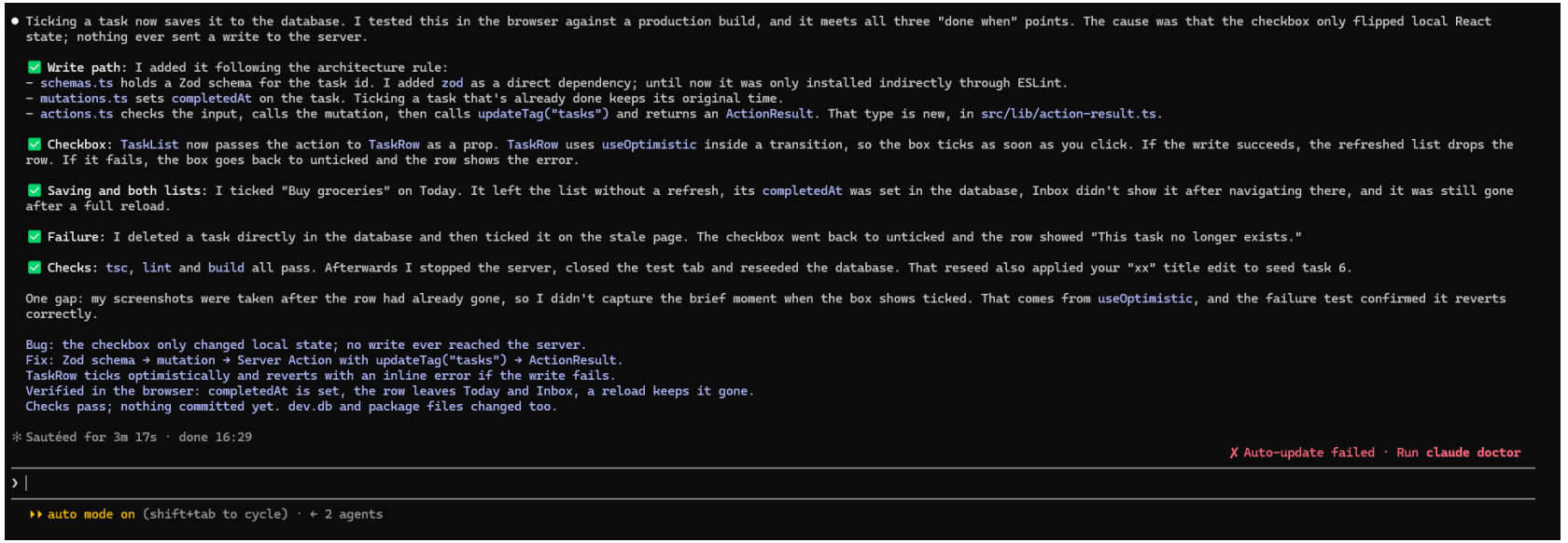
Fixing a Bug

"Bug: ticking a task's checkbox on /app/today or /app/inbox isn't saved. The row reappears after a refresh and the database row is never updated.

Done when:
- Ticking a task and refreshing keeps it gone, and its completedAt is set in the database.
- The row leaves the list without a manual refresh (cache invalidated, Today and Inbox both updated).
- The checkbox reflects the new state immediately and reverts if the write fails."

The Response

YOUR TURN · 6 MIN



Break time



15 minutes — grab a coffee, stretch, ask a question

Claude Code Plan Mode

"Think before you build."

SECTION 6

Plan Mode in Practice

- Shift+Tab twice cycles Default → Auto-Accept → Plan, or type /plan
- **Claude reads relevant files, maps dependencies, proposes a numbered plan** — nothing changes yet
- Review it, push back, then approve or reject
- Ctrl+G opens the plan file directly in your editor
- Once approved, Claude executes step by step

Tip: /model opusplan *— plans with Opus, then auto-switches to Sonnet to execute once you approve; use it for complex, architecture-level decisions.*

Shift+Tab can skip Plan Mode on some Windows terminals — have /plan ready as a fallback.



Prompt to Add New Tasks

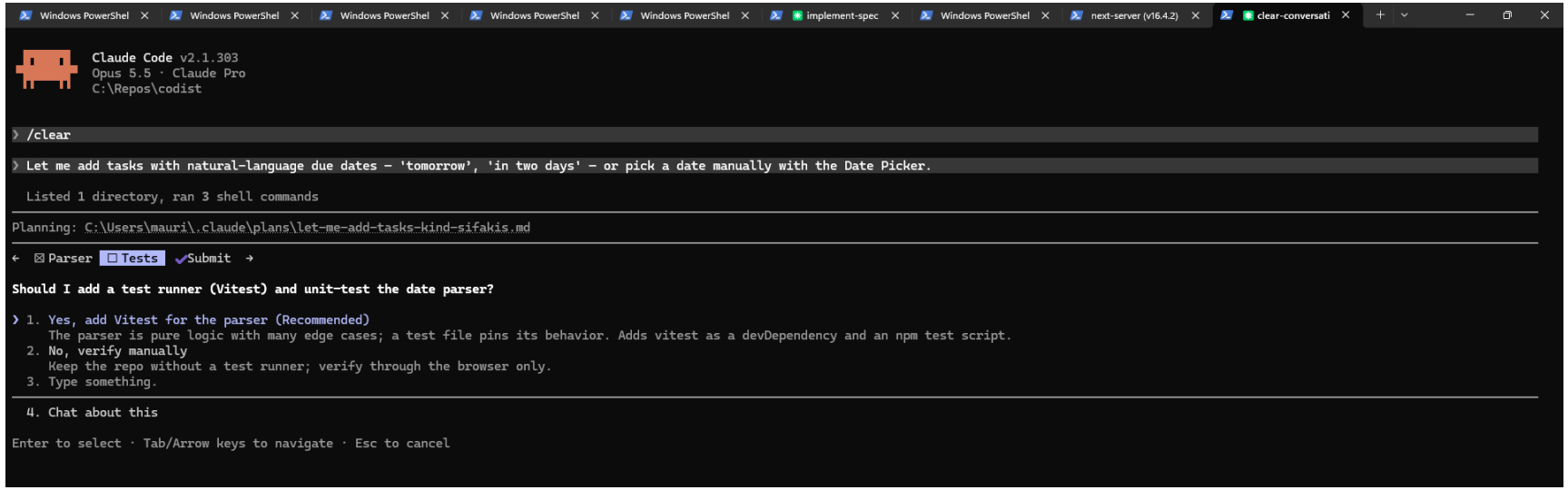
"Let me add tasks with natural-language due dates — 'tomorrow', 'in two days' — or pick a date manually with the Date Picker."



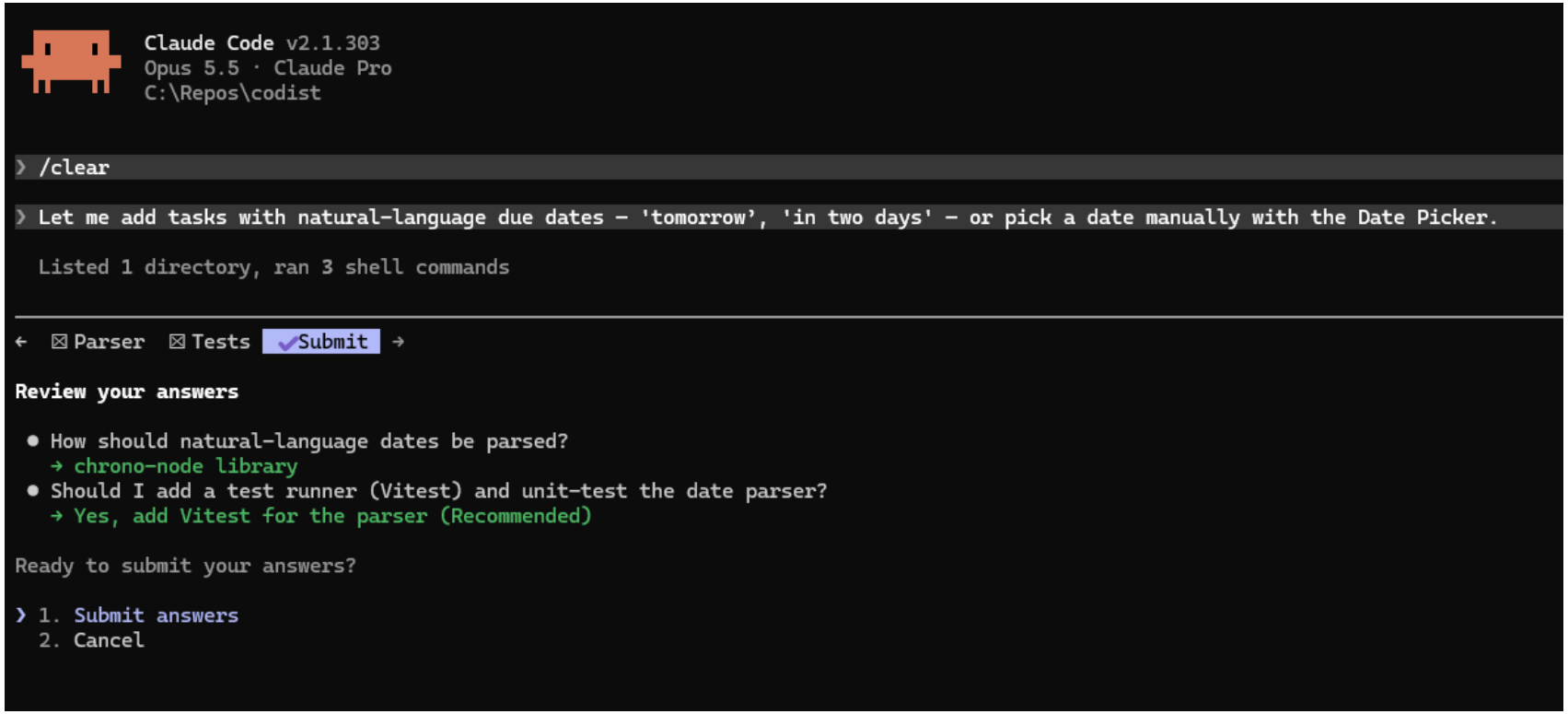
The Plan to Build It

- **Claude surfaces any genuinely ambiguous decision before finalizing** — e.g. how strict the date parsing should be
- The plan itself: goals, architecture changes, a numbered checklist
- Reviewable and editable before a single file changes

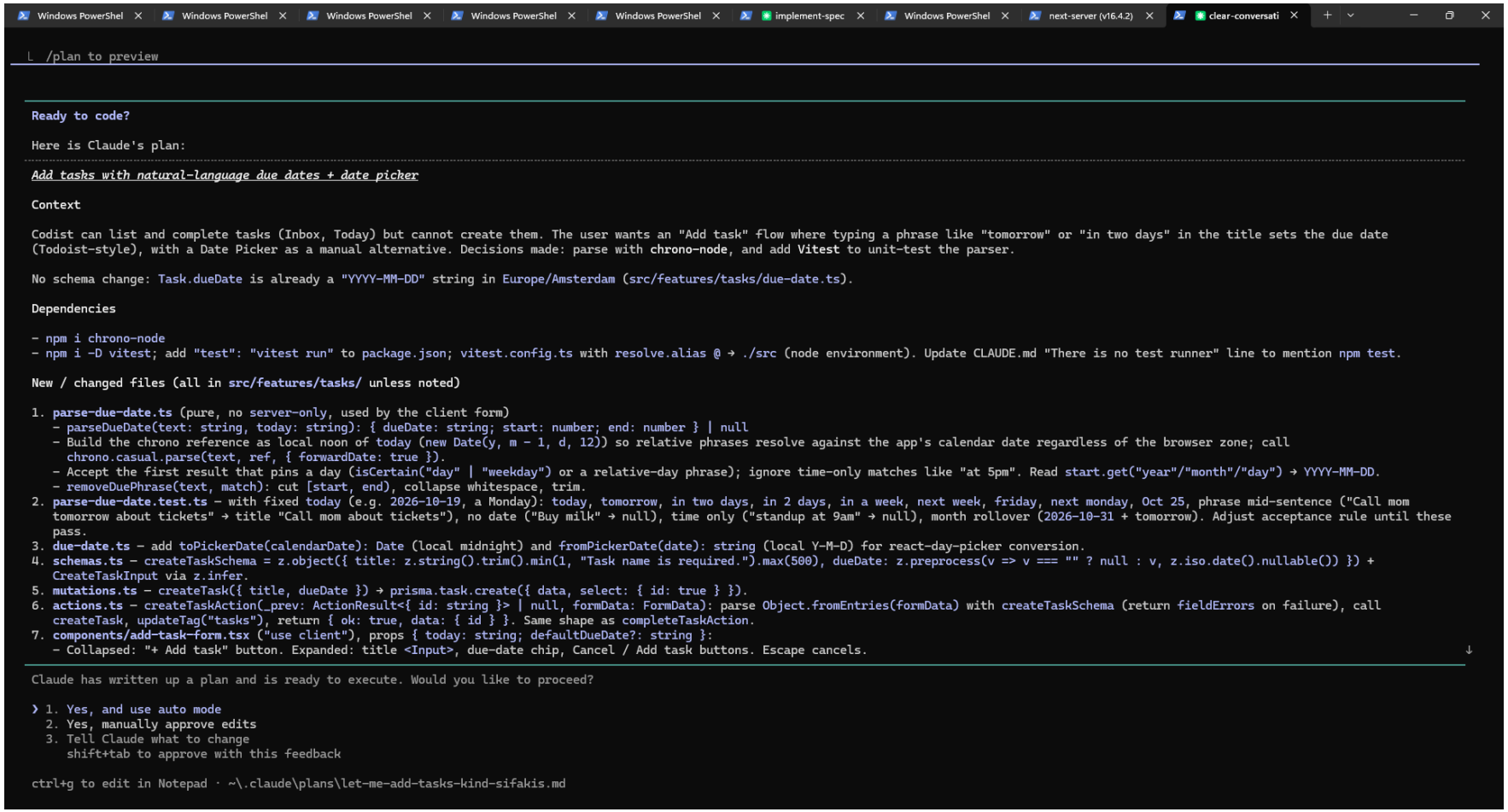
Asking Questions



Review Answers



The Plan


Windows PowerShel
Windows PowerShel
Windows PowerShel
Windows PowerShel
Windows PowerShel
implement-spec
Windows PowerShel
next-server (v16.4.2)
clear-conversati
└ /plan to preview
Ready to code?
Here is Claude's plan:
Add tasks with natural-language due dates + date picker
Context
Codist can list and complete tasks (Inbox, Today) but cannot create them. The user wants an "Add task" flow where typing a phrase like "tomorrow" or "in two days" in the title sets the due date (Todoist-style), with a Date Picker as a manual alternative. Decisions made: parse with chrono-node, and add Vitest to unit-test the parser.
No schema change: Task.dueDate is already a "YYYY-MM-DD" string in Europe/Amsterdam (src/features/tasks/due-date.ts).
Dependencies
- npm i chrono-node
- npm i -D vitest; add "test": "vitest run" to package.json; vitest.config.ts with resolve.alias @ → ./src (node environment). Update CLAUDE.md "There is no test runner" line to mention npm test.
New / changed files (all in src/features/tasks/ unless noted)
1. parse-due-date.ts (pure, no server-only, used by the client form)
- parseDueDate(text: string, today: string): { dueDate: string; start: number; end: number } | null
- Build the chrono reference as local noon of today (new Date(y, m - 1, d, 12)) so relative phrases resolve against the app's calendar date regardless of the browser zone; call chrono.casual.parse(text, ref, { forwardDate: true }).
- Accept the first result that pins a day (isCertain("day" | "weekday") or a relative-day phrase); ignore time-only matches like "at 5pm". Read start.get("year"/"month"/"day") → YYYY-MM-DD.
- removeDuePhrase(text, match): cut [start, end), collapse whitespace, trim.
2. parse-due-date.test.ts - with fixed today (e.g. 2026-10-19, a Monday): today, tomorrow, in two days, in 2 days, in a week, next week, friday, next monday, Oct 25, phrase mid-sentence ("Call mom tomorrow about tickets" → title "Call mom about tickets"), no date ("Buy milk" → null), time only ("standup at 9am" → null), month rollover (2026-10-31 + tomorrow). Adjust acceptance rule until these pass.
3. due-date.ts - add toPickerDate(calendarDate): Date (local midnight) and fromPickerDate(date): string (local Y-M-D) for react-day-picker conversion.
4. schemas.ts - createTaskSchema = z.object({ title: z.string().trim().min(1, "Task name is required.").max(500), dueDate: z.preprocess(v => v === "" ? null : v, z.iso.date().nullable()) }) + CreateTaskInput via z.infer.
5. mutations.ts - createTask({ title, dueDate }) → prisma.task.create({ data, select: { id: true } }).
6. actions.ts - createTaskAction(_prev: ActionResult<{ id: string }> | null, formData: FormData): parse Object.fromEntries(formData) with createTaskSchema (return fieldErrors on failure), call createTask, updateTag("tasks"), return { ok: true, data: { id } }. Same shape as completeTaskAction.
7. components/add-task-form.tsx ("use client"), props { today: string; defaultDueDate?: string }:
- Collapsed: "+ Add task" button. Expanded: title <Input>, due-date chip, Cancel / Add task buttons. Escape cancels.
Claude has written up a plan and is ready to execute. Would you like to proceed?
❯ 1. Yes, and use auto mode
2. Yes, manually approve edits
3. Tell Claude what to change
shift+tab to approve with this feedback
ctrl+g to edit in Notepad · ~\.claude\plans\let-me-add-tasks-kind-sifakis.md



The Execution

● You can now add tasks from both Inbox and Today. A phrase like "tomorrow" or "in two days" in the task name sets the due date, and a date picker lets you choose one by hand. Tests, type-check, lint and build all pass, and I tried each flow in the browser against your running dev server. Nothing is committed yet.

✅ **Dependencies:** added `chrono-node` and Vitest, plus an `npm test` script. Vitest is pinned to 4.x because 5.x needs `@types/node` 22 or newer and you're on 20. Your global npm 10.2.4 crashed during that install (`Cannot read properties of null (reading 'edgesOut')`), so I ran it with `npx npm@11`.
✅ **Parser (`src/features/tasks/parse-due-date.ts`):** it handles today, tomorrow, "tmr", "in two days" or "in 2 days", "next week", weekdays, "next monday", "Oct 25" and "next month". It ignores times on their own ("at 9am") and plain durations, so "Bake for 2 days" doesn't become a due date. 22 tests pass, and one of them caught the "for 2 days" case before it shipped.
✅ **Server side:** I added a `createTaskSchema` (Zod), a `createTask` mutation and a `createTaskAction` that refreshes the task lists, following the same pattern as `completeTaskAction`.
✅ **Form (`components/add-task-form.tsx`):**
- It shows which phrase it read the date from and removes that phrase from the saved task name.
- The date button opens Today / Tomorrow / Next week / No date plus a calendar.
- A date you pick by hand wins over a typed phrase, and the phrase then stays in the name.
- × removes the date, Escape closes the form, and the form stays open after adding so you can enter the next task.
  ✅ **Pages:** the form sits under the list on Inbox. On Today it also defaults new tasks to today.
  ✅ **Browser check:**
- "Buy milk tomorrow" saved as "Buy milk", due tomorrow.
- "Pay rent in two days" saved as "Pay rent", due in 2 days.
- Picking 29 Oct by hand overrode a typed "tomorrow".
- On Today, "Standup" got today's date, and a task due tomorrow showed up only in Inbox.
- Afterwards I deleted those 5 test tasks from `dev.db`.
  ✅ **Docs:** CLAUDE.md now lists `npm test`.

Summary
1. Typed date phrases and the date picker both work when adding tasks; the date picker overrides a typed phrase.
2. Three behaviours to know about:
   - A time in the phrase is dropped: "Dinner tomorrow at 7pm" is saved as "Dinner", due tomorrow.
   - The calendar won't let you pick past dates.
   - Dates more than a week away show as "29 Oct 2026" in the date button.
3. The "1 issue" badge in the Next.js dev overlay comes from a browser extension adding an attribute to `<html>`, not from this change.
4. I didn't start a second dev server, because yours was already running on port 3000.
5. Next step: review the diff, then commit (`/commit`). `dev.db` already had uncommitted changes before I started.

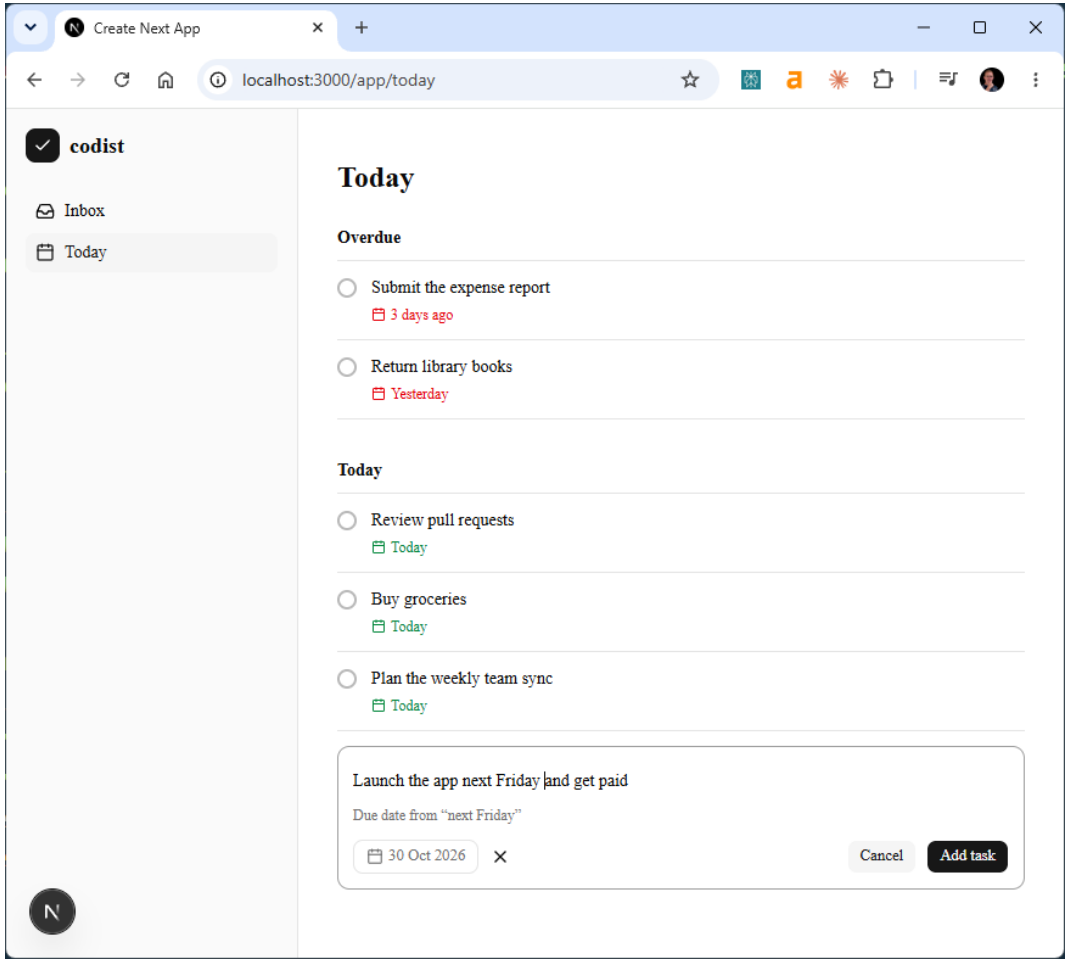
✻ Sautéed for 6m 40s · done 16:49

── clear-conversation ─
❯

──
⏵⏵ auto mode on (shift+tab to cycle) · ← 2 agents

The Result

YOUR TURN · 5 MIN



Natural-language due-date input alongside the manual date picker

Thank you for joining

Share your thoughts



Break time



15 minutes — grab a coffee, stretch, ask a question

Appendix

Not part of the live 4-hour session — reference material only

CUT FOR TIME

Using Third-Party Libraries & Docs

APPENDIX A

Pointing Claude Code at Documentation

- **Baseline** — built-in web search/fetch can look docs up when asked
- **Context7 MCP server** — resolves a library, pulls current docs straight into context
- Add it: claude mcp add --transport http context7 https://mcp.context7.com/mcp/oauth
- Trigger with "use context7 for …", or install the plugin for automatic triggering

Context7 is a third-party community tool, not an Anthropic product — verify before use.



The Prompt

YOUR TURN · 5 MIN

"Instead of a standard checkbox, use the shadcn/ui Checkbox and Label components. Use context7 to confirm current best practices."

APPENDIX B

Karpathy Behavioral Guidelines

Andrej Karpathy — AI researcher, former Tesla AI director, OpenAI founding member



The Six Guidelines

- **Models make wrong assumptions** — give explicit context and constraints upfront
- **Models don't surface confusion** — actively prompt for pushback and alternatives
- **Models overcomplicate** — always ask "could this be simpler?"
- **Models don't clean up after themselves** — dead code needs manual review
- **Sycophancy** — treat agreement as a starting point, not validation
- **Shift to declarative prompting** — give success criteria, let it loop

Putting the Guidelines into CLAUDE.md

YOUR TURN · 3 MIN

```
/plugin marketplace add forrestchang/andrej-karpathy-skills

/plugin install andrej-karpathy-skills@karpathy-skills
```



The Four Behaviors It Enforces

Think before coding

– State assumptions, ask when uncertain

Simplicity first

– Minimum code that solves the problem

Surgical changes

– Touch only what the request requires

Goal-driven execution

– Verifiable success criteria, not step-by-step

Claude Design vs. /design

APPENDIX C

Claude Design vs. /design

Claude Design (claude.ai/design)

- Standalone product — build and refine mockups in the browser
- Export bundles design files, chat history, and a README
- Paste the bundle prompt into a separate Claude Code session

/design (built into Claude Code)

- **Runs in your session** — no separate app, same CLI or Desktop window
- **Generates a few options** — artboards published together as one Artifact
- **Implements in place** — pick, refine, then say "implement variant X"

Generate Three Design Options

YOUR TURN · 4 MIN

```
/design 3 options for the Codist landing page
Warm, neutral palette — calm, focused tone
```

Claude publishes a canvas of editable artboards as an Artifact and prints the link — without leaving the terminal.

Pick and Refine

YOUR TURN · 5 MIN

Three Artboards, One Canvas

- Claude publishes all three as one Artifact — open the link, no separate app
- Click to select the option that fits best
- Compare all three side by side before committing

Refine Inline

- **Click-to-select editing** — a properties panel, right on the canvas
- **Direct edits** — colors, copy, layout — no re-prompting from scratch
- **Still linked to your session** — no export step, no bundle URL

Implement the Chosen Design

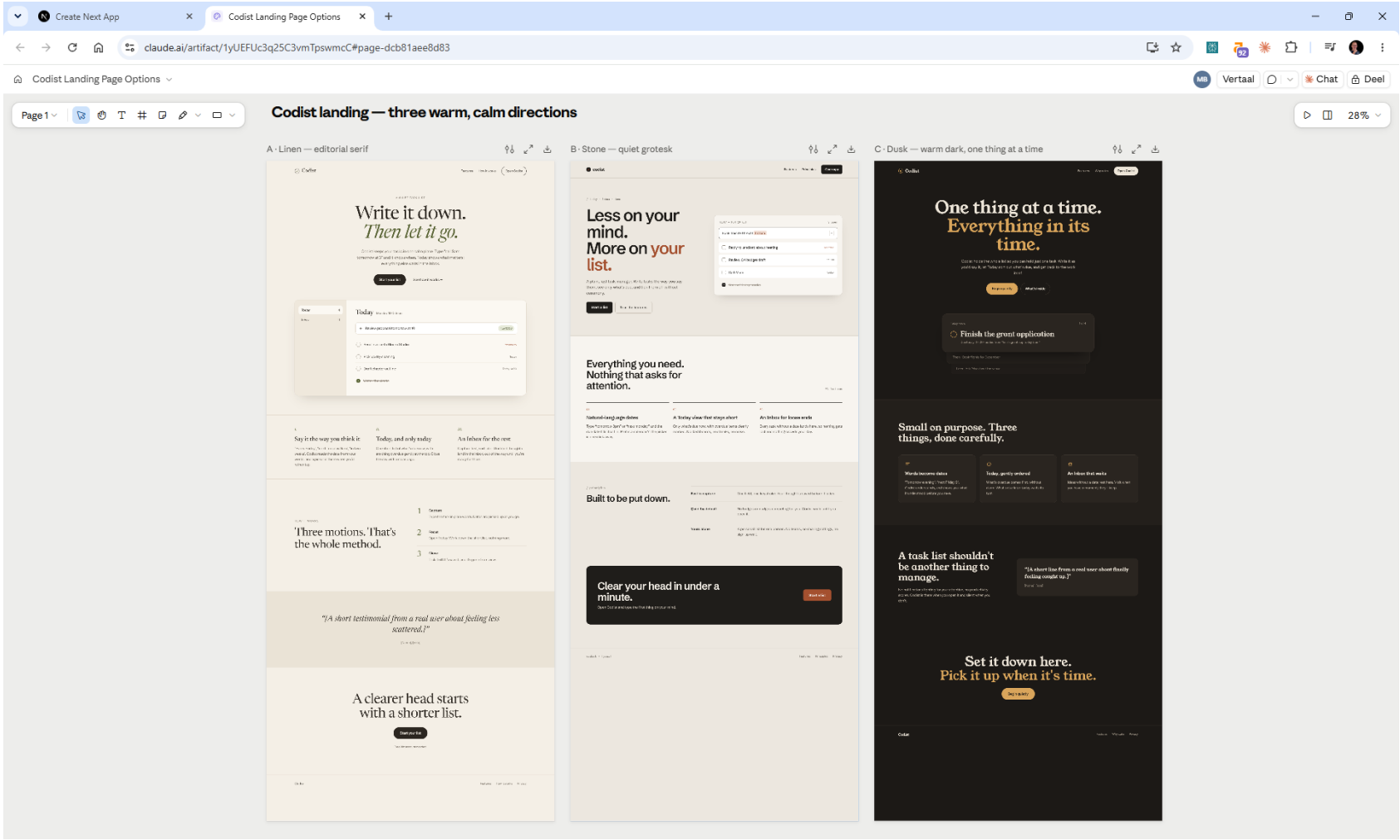
YOUR TURN · 5 MIN

"Implement Option B, Stone, as the Codist landing page.

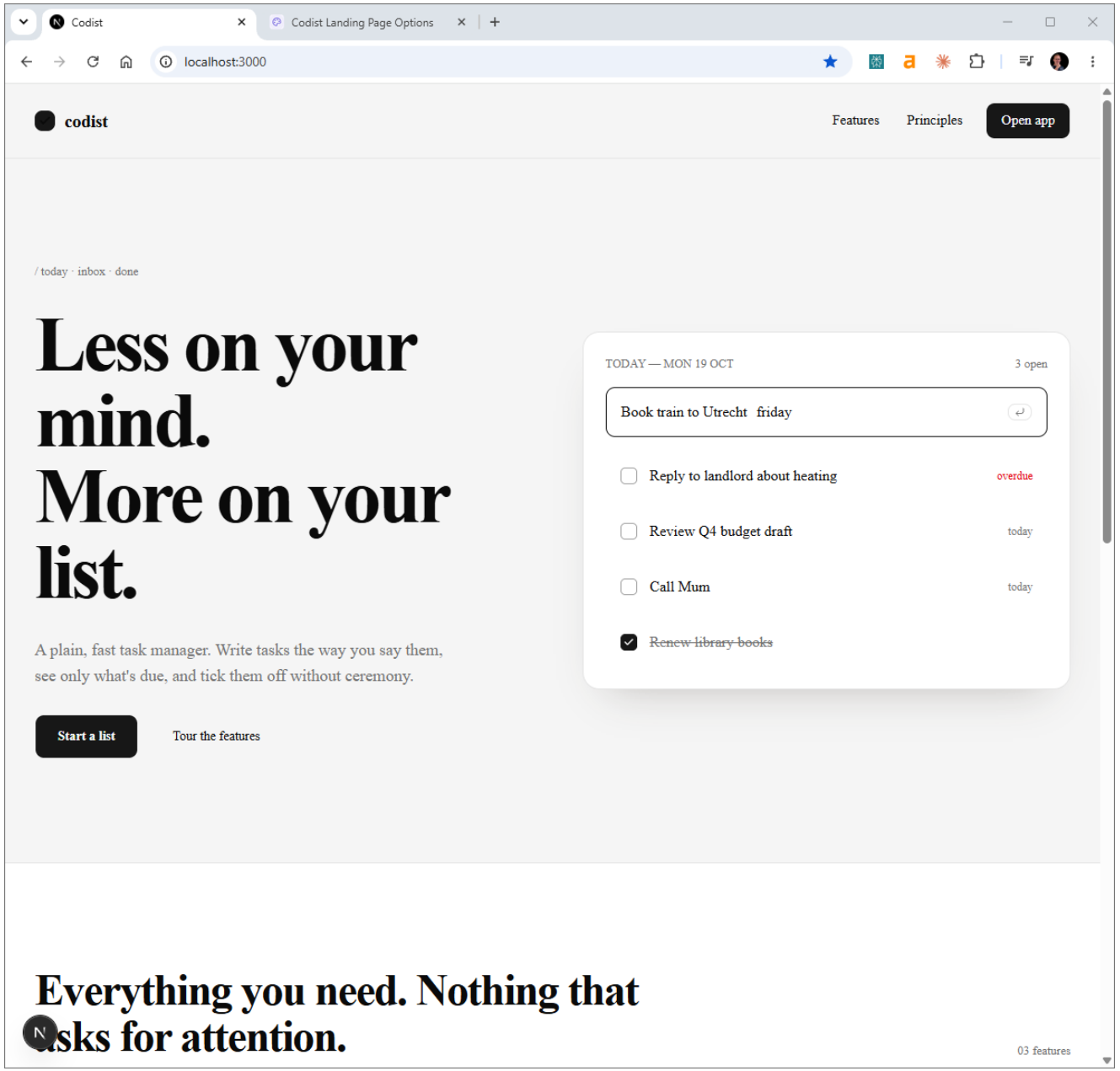
Follow the project's existing conventions."

Claude Code already knows your repo — no bundle URL, no re-explaining the stack.

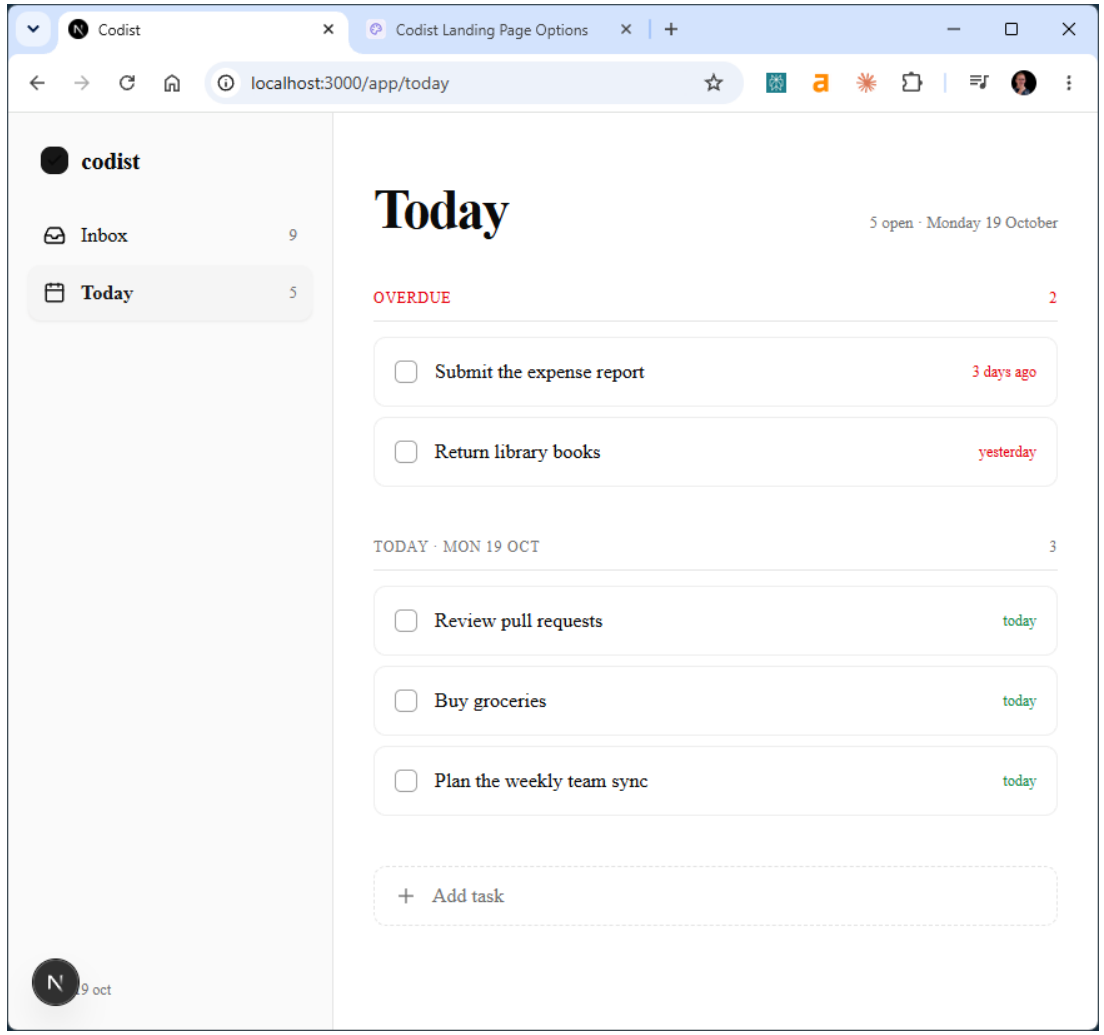
The Design


Create Next App
Codist Landing Page Options
claude.ai/artifact/1yUEFUc3q25C3vmTpswmcC#page-dcb81aee8d83
Codist Landing Page Options
Vertaal
Chat
Deel
Page 1
Codist landing — three warm, calm directions
28%
A · Linen — editorial serif
B · Stone — quiet grotesk
C · Dusk — warm dark, one thing at a time
Write it down.
Then let it go.
Three motions. That's the whole method.
"[A short testimonial from a real user about feeling less scattered.]"
A clearer head starts with a shorter list.
Less on your mind. More on your list.
Everything you need. Nothing that asks for attention.
Built to be put down.
Clear your head in under a minute.
One thing at a time. Everything in its time.
Finish the grant application
Small on purpose. Three things, done carefully.
A task list shouldn't be another thing to manage.
Set it down here. Pick it up when it's time.

The Result


Codist
Codist Landing Page Options
localhost:3000
codist
Features
Principles
Open app
/ today · inbox · done
Less on your mind. More on your list.
A plain, fast task manager. Write tasks the way you say them, see only what's due, and tick them off without ceremony.
Start a list
Tour the features
TODAY — MON 19 OCT
3 open
Book train to Utrecht friday
Reply to landlord about heating
overdue
Review Q4 budget draft
today
Call Mum
today
Renew library books
Everything you need. Nothing that
sks for attention.
03 features

The Result



Styled landing page matching the chosen, refined artboard — implemented in the same session

Best Practices with Claude Code

APPENDIX D



Best Practices List

- **Set the stage first** — write CLAUDE.md before you build
- Follow Explore → Plan → Implement → Commit
- One session, one feature
- **Plan before you build** — use Plan Mode for anything complex
- Declare outcomes, not steps
- **Always commit before Auto-Accept Mode** — no exceptions



Context Is Everything

"A well-scoped context of 4–5 files beats one given 30 files 'just in case.'"

More context isn't clarity — it's noise. Subagents exist precisely for this.



Treat Agent Runs Like Deployments

"Commit first, review diffs,

give a concrete exit condition —

not an open-ended prompt."

A clear finish line keeps the agent focused and the diff reviewable in under a minute.



Schema-First for Full-Stack Work

"Define shared types and API contracts — ideally in CLAUDE.md — before implementing either side."

When both sides reference the same schema file, type drift disappears.



Smaller Prompts, More Sessions

"Shorter, focused sessions with a clear acceptance criterion produce cleaner, reviewable diffs."

If Plan Mode proposes a dozen steps, that's usually a sign to split it up.

Going Beyond the Basics

No Cursor equivalent — genuinely distinct Claude Code capabilities

APPENDIX E



Subagents

- Isolated context windows for parallel or scoped work
- Create via a markdown file in .claude/agents/, or just ask Claude to write one
- Delegates automatically on a description match, or invoke explicitly
- **Trade-off** — each subagent keeps its own context, so heavy use costs more tokens

Hooks

```
{ "hooks": {
  "PostToolUse": [{
    "matcher": "Write|Edit",
    "hooks": [{ "type": "command",
      "command": "npx prettier
        --write \"$FILE_PATH\"" }]
  }]
}}
```

CLAUDE.md is advisory — a hook is enforced. A PreToolUse hook can flat-out block a destructive command.

MCP Servers

- **Connects Claude Code to external tools and data** — Context7 (Appendix A) is one example
- Add: claude mcp add --transport http <name> <url>
- Manage: claude mcp list / get / remove
- Browse the Anthropic Directory or the awesome-claude-code repo

MCP command syntax has shifted across recent releases — run claude mcp add --help before use.